Reliability Architecture for AI Coding Agent Harnesses

First Author[1], Second Author[2], Third Author[2], Fourth Author[1] and Fifth Author[1]
[1]Pragnition Labs, [*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

Multi-step AI agent pipelines face a fundamental reliability challenge: per-step success rates compound multiplicatively, producing exponential decay in end-to-end reliability. At 95% per-step accuracy across 20 steps, pipeline success is 36%. This paper develops a formal framework for reliability architecture in such pipelines, drawing on classical reliability theory, optimal inspection scheduling, and information-theoretic verification bounds.

We contribute four formal results: (1) a compound error sensitivity theorem showing that a 1% relative improvement in per-step reliability yields an n% relative improvement end-to-end, where n is the pipeline length; (2) an optimal verification scheduling formulation via dynamic programming, recovering the Young-Daly checkpoint interval as a special case and proving that three verification rounds capture over 96% of detectable errors; (3) a formal model of circular validation bias showing that same-author code and tests share blind spots, with empirical calibration against the METR finding that roughly 50% of test-passing PRs would not be merged; and (4) a cost-of-failure threshold theorem establishing when structural enforcement (deterministic, agent-proof constraints) becomes cost-effective over prompt-based instructions.

We calibrate these models against published benchmarks (SWE-bench, RE-Bench, METR evaluations, AgentSpec) and derive practical design principles: a four-layer verification architecture ordered by efficiency, adaptive verification intensity based on risk and information gap, and the structural enforcement boundary separating deterministic from probabilistic quality assurance. The framework provides engineering guidance for building agent systems that degrade gracefully rather than catastrophically as pipeline complexity grows.

1. Introduction

The deployment of large language model (LLM) agents on multi-step software engineering tasks has revealed a stark quantitative reality: compound errors destroy end-to-end reliability. At 85% per-step accuracy across 10 steps, the probability that every step succeeds is $0.85^{10} = 19.7\%$ (Rajan, 2026). At 90% accuracy across 20 steps, it falls to 12%. Field measurements confirm these predictions: Hariharan et al. (2025) report 63% failure rates on 100-step tasks, and METR found that roughly 50% of SWE-bench test-passing pull requests would not be merged by repository maintainers (METR, 2026).

This is the single most important quantitative fact about agent workflows. No amount of prompt engineering, context optimization, or architectural governance matters if compound errors destroy the output. Yet the field lacks a formal framework for reasoning about when, where, and how aggressively to verify agent output.

This paper develops such a framework. This is a *theory and position paper*: we develop formal tools and apply them to derive design principles, but we do not conduct new experiments. Empirical calibration draws on published benchmarks and studies. We note that this is a rapidly evolving area where practitioner discourse (blog posts, industry reports, and preprints) frequently precedes peer-reviewed publication; we draw on such sources where they provide the best available evidence, and flag them accordingly. Our contributions are:

1. **Formal compound error model** (§3): We model agent pipelines as series reliability systems, prove the compound error sensitivity theorem (elasticity equals pipeline length), and extend the model to correlated failures via the Marshall-Olkin common-cause framework. We calibrate against published benchmarks and show that the independent p^n model is a reasonable first approximation, but that error clustering in problem-framing steps dominates execution-step errors.
2. **Optimal verification scheduling** (§4): We formulate checkpoint placement as a dynamic programming problem, prove equal-spacing optimality for homogeneous pipelines, derive the optimal checkpoint count as a discrete Young-Daly analog, and establish the geometric diminishing returns of successive verification rounds. Calibration against Hariharan et al. (2025) confirms that three rounds are optimal.
3. **Circular validation bias** (§5): We formalize the bias β that arises when the same agent produces both code and tests, model it via blind-spot sets, and connect it to the METR merge-gap finding. We prove that self-verification provides zero bias reduction and that cross-family verification is where gains concentrate.
4. **Structural enforcement boundary** (§6): We define the distinction between structural enforcement (deterministic, agent-proof) and prompt enforcement (probabilistic), derive the cost-of-failure threshold L^* above which structural enforcement is cost-effective, and show that this threshold drops as $1/n$ for multi-step pipelines.
5. **Adaptive verification architecture** (§7): We synthesize these results into a four-layer verification architecture with formal efficiency ordering, derive switching thresholds for adaptive verification intensity, and address the economics of multi-agent verification (including the sequential reasoning penalty documented by Google Research).

2. Preliminaries and Definitions

Definition 2.1 (Agent Pipeline)**.** *An* n-step agent pipeline *is a tuple* $\mathcal{P} = (S_1, S_2, \ldots, S_n)$ *where each* S_i *is a stochastic transformation mapping input artifact* a_{i-1} *to output artifact* a_i*. The pipeline is a* series reliability system *with structure function* $\phi(x_1, \ldots, x_n) = \prod_{i=1}^{n} x_i$*, where* $x_i \in \{0, 1\}$ *indicates whether step* i *produced correct output.*

Definition 2.2 (Per-Step Success Probability)**.** *For step* i*, define* $p_i = \mathbb{P}(x_i = 1 \mid a_{i-1}$ *is correct*$)$*.*

Definition 2.3 (Error Propagation Probability)**.** *Define* $q_i = \mathbb{P}(x_i = 1 \mid a_{i-1}$ *is incorrect*$)$*, the probability that step* i *produces correct output despite incorrect input. In most agent workflows,* $q_i \ll p_i$*.*

Definition 2.4 (Verification Layer)**.** *A* verification layer ℓ *is characterized by detection rate* $d(\ell) = \mathbb{P}($*flag* $\mid$ *defect*$)$*, false positive rate* $f(\ell) = \mathbb{P}($*flag* $\mid$ *no defect*$)$*, cost* $c(\ell)$*, and latency* $\tau(\ell)$*. Its* efficiency *is* $\eta(\ell) = d(\ell)/c(\ell)$*.*

3. Compound Error Dynamics

3.1. The Series Reliability Product Law

Theorem 3.1 (Series Reliability)**.** *If step outcomes are conditionally independent given correct inputs at each stage, the end-to-end reliability is*

$$R(n) = \prod_{i=1}^{n} p_i \tag{1}$$

Proof. By the structure function $\phi(\mathrm{x}) = \prod x_i$, the pipeline succeeds iff all $x_i = 1$. Under conditional independence and the chain rule: $R(n) = \mathbb{P}\left(\bigcap_{i=1}^{n}\{x_i = 1\}\right) = \prod_{i=1}^{n} \mathbb{P}(x_i = 1 \mid x_1 = \cdots = x_{i-1} = 1) = \prod_{i=1}^{n} p_i$. □

Corollary 3.1 (Homogeneous Pipeline). *If $p_i = p$ for all i, then $R(n) = p^n$, which decays exponentially:*

p	n=5	n=10	n=20	n=50
0.99	0.951	0.904	0.818	0.605
0.95	0.774	0.599	0.358	0.077
0.90	0.590	0.349	0.122	0.005

3.2. The Compound Error Sensitivity Theorem

Theorem 3.2 (Sensitivity of Pipeline Reliability). *For a homogeneous pipeline with $R(n) = p^n$:*

$$\frac{\partial R}{\partial p} = n \cdot p^{n-1} \tag{2}$$

The elasticity of pipeline reliability with respect to per-step reliability is exactly n: a 1% relative improvement in per-step reliability yields an n% relative improvement end-to-end.

Proof. Direct differentiation gives $dR/dp = np^{n-1}$. The elasticity $\varepsilon = (p/R)(dR/dp) = (p \cdot np^{n-1})/p^n = n$. □

Corollary 3.2 (Weakest-Link Priority). *For a heterogeneous pipeline, $\partial R/\partial p_j = R/p_j$, which is inversely proportional to p_j. Improvement effort should target the step with the lowest reliability.*

Example 3.1. *For $p_0 = 0.95$ and $n = 10$: $\partial R/\partial p = 10 \times 0.95^9 = 6.30$. A 1-percentage-point increase in per-step accuracy (from 0.95 to 0.96) raises pipeline reliability from 59.9% to 66.5%, a gain of 6.6 pp.*

3.3. Correlated Failures: The Common-Cause Model

The independence assumption is unrealistic. Agent pipelines exhibit correlated failures: task misunderstanding at step 1 biases all subsequent steps; shared model weights induce common-mode failures; context drift affects multiple steps simultaneously.

Definition 3.1 (Common-Cause Failure Model). *Following the Marshall-Olkin framework (Marshall and Olkin, 1967), decompose each step's failure into an independent component $Z_i \sim \text{Bernoulli}(\alpha_i)$ and a shared common-cause component $Y \sim \text{Bernoulli}(\gamma)$:*

$$X_i = \max(Z_i, Y) \tag{3}$$

where X_i is the failure indicator for step i.

Theorem 3.3 (Correlated Failure Reliability). *Under the common-cause model, pipeline reliability is:*

$$R(n) = (1-\gamma)\prod_{i=1}^{n}(1-\alpha_i) \tag{4}$$

The common-cause failure rate γ imposes a hard ceiling: no amount of per-step improvement can raise $R(n)$ above $(1-\gamma)$.

Proof. The pipeline succeeds iff $Y = 0$ (no common-cause failure) and $Z_i = 0$ for all i. Since Y and all Z_i are independent: $R(n) = (1-\gamma)\prod_{i=1}^{n}(1-\alpha_i)$. □

Remark. *This decomposition is practically important. Research on agent failures suggests that most failures are attributable to problem framing and context management (the γ component), not execution-step errors (the α_i components) (Cemri et al., 2025). Improving individual α_i has limited effect when γ dominates; the leverage is in reducing γ through better context engineering and specification.*

3.4. Empirical Calibration

We calibrate the model against published benchmarks by back-solving for effective per-step reliability:

Source	Setting	n	R_{obs}	Implied p
Rajan (2026)	General agent	20	0.12	0.900
METR (METR, 2026)	SWE-bench merge	10	0.50	0.933
Devin (Cognition Labs, 2025)	PR merge (2025)	12	0.67	0.967
RE-Bench (Stanford HAI, 2025)	Long-horizon	100	0.37	0.990

The implied per-step reliabilities (93–99%) are internally consistent. We emphasize that this calibration demonstrates *consistency*, not *validation*: the model predicts $R = p^n$, and the calibration assumes the same formula to back-solve for p. Independent measurements of both p and R under controlled conditions would be needed for true validation; such data does not yet exist at scale. The RE-Bench figure (99.0%) is higher because many steps in long-horizon tasks are trivial; the failure risk concentrates in a small fraction of steps, consistent with the common-cause model (Section 3.1).

The heterogeneous-step model provides sharper prescriptions. With $p_1 = 0.80$ (problem framing), $p_{2\ldots9} = 0.98$ (execution), $p_{10} = 0.92$ (integration) for a 10-step pipeline: $R = 0.80 \times 0.98^8 \times 0.92 = 0.626$. Improving p_1 from 0.80 to 0.90 yields $R = 0.703$ (12.3% absolute gain); improving all execution steps from 0.98 to 0.99 yields $R = 0.662$ (5.7% gain at 8× the intervention breadth).

4. Optimal Verification Scheduling

4.1. Problem Formulation

Definition 4.1 (Verification Scheduling Problem)**.** *Given an n-step pipeline with per-step failure probabilities $\epsilon_i = 1 - p_i$, verification cost c_v, detection probability v, and rework cost $c_r(d)$ increasing with delay d, find the verification set $V^* \subseteq \{1, \ldots, n\}$ minimizing total expected cost.*

We adopt a segment-based formulation. Partition the pipeline into k contiguous segments separated by verification checkpoints. Segment j spans steps $[a_j, b_j]$ with failure probability $F_j = 1 - \prod_{i=a_j}^{b_j} p_i$ and length $L_j = b_j - a_j + 1$.

The expected cost given k segments with verification after each internal boundary:

$$\mathbb{E}[\text{Cost}] = (k-1) \cdot c_v + \sum_{j=1}^{k} g(L_j) \tag{5}$$

where $g(L) = (1-p^L) \cdot c_0 \cdot L$ is the expected rework cost for a segment of length L (in the homogeneous case).

4.2. Equal-Spacing Optimality

Theorem 4.1 (Equal-Spacing Optimality)**.** *For a homogeneous pipeline ($p_i = p$), convex rework cost, uniform detection probability, and a fixed checkpoint budget of $k-1$, the cost-minimizing partition divides the pipeline into k equal segments of length n/k.*

Proof sketch. The segment cost $g(L) = (1-p^L) \cdot c_0 \cdot L$ is convex in L for $0 < p < 1$, $L > 0$. To verify, write $g(L) = c_0(L - L \cdot p^L)$. Then:

$$g'(L) = c_0\left[1 - p^L - Lp^L \ln p\right] = c_0\left[1 - p^L(1 + L\ln p)\right]$$
$$g''(L) = c_0\left[-p^L \ln p(1 + L\ln p) - p^L \ln p\right] = c_0\left[-p^L \ln p(2 + L\ln p)\right]$$

Since $\ln p < 0$ for $p < 1$, let $\mu = -\ln p > 0$. Then $g''(L) = c_0 \mu p^L(2 - \mu L)$. For $L < 2/\mu$ (i.e., $L < 2/|\ln p|$), this is positive. At $p = 0.95$, $2/\mu \approx 39$, so g is convex for all segment lengths up to 39 steps, covering practical pipelines. By Schur-convexity of a sum of convex functions subject to $\sum L_j = n$, the minimum is at the equal-length partition. □

Theorem 4.2 (Optimal Checkpoint Count)**.** *The optimal number of segments satisfies:*

$$k^* \approx \left\lfloor \sqrt{\frac{n(1-p)c_0}{c_v}} \right\rfloor, \qquad L^* = n/k^* \approx \sqrt{\frac{c_v}{(1-p)c_0}} \tag{6}$$

This is a discrete analog of the Young-Daly checkpoint interval formula (Daly, 2006; Young, 1974).

Example 4.1. *For $p = 0.95$, $c_v = 1$, $c_0 = 5$, $n = 20$: $L^* = \sqrt{1/(0.05 \times 5)} = 2$, suggesting verification every 2 steps. For $p = 0.99$, $c_v = 10$: $L^* = \sqrt{10/(0.01 \times 5)} \approx 14$, suggesting verification once or twice in a 20-step pipeline.*

4.3. Diminishing Returns of Verification Rounds

Theorem 4.3 (Geometric Diminishing Returns)**.** *Let $M(k)$ be the expected number of errors detected by k verification rounds, each independently detecting remaining errors with probability v. Then:*

$$\Delta M(k) = M(k) - M(k-1) = N_0 \cdot v \cdot (1-v)^{k-1} \tag{7}$$

The marginal value of round k decays geometrically with ratio $(1-v)$.

Proof. After k rounds, the probability a given error remains undetected is $(1-v)^k$. Hence $M(k) = N_0(1-(1-v)^k)$ and $\Delta M(k) = N_0 v(1-v)^{k-1}$. □

Empirical calibration. Hariharan et al. (2025) report cumulative error detection of 62% after round 1, 89% after round 2, and 96.5% after round 3. Fitting the geometric model with $v = 0.62$:

Round k	Predicted $D(k)$	Observed $D(k)$	Residual
1	0.620	0.620	0.000
2	0.856	0.890	−0.034
3	0.945	0.965	−0.020

The slight underprediction at rounds 2–3 is consistent with an improving detection rate ($v_1 = 0.62$, $v_2 = 0.71$, $v_3 = 0.68$), where the verifier sharpens its focus after eliminating easy errors. The stopping criterion (verify until marginal expected catch falls below verification cost) yields $k \approx 3$ for typical parameters, confirming the empirical finding.

5. Circular Validation Bias

5.1. Formal Definition

Definition 5.1 (Circular Validation Bias)**.** *For a code artifact P and test suite T, the* circular validation bias *is:*

$$\beta = \mathbb{P}(T \text{ passes} \mid P \text{ buggy, same author}) - \mathbb{P}(T \text{ passes} \mid P \text{ buggy, different author}) \tag{8}$$

5.2. The Blind-Spot Model

Definition 5.2 (Blind-Spot Set)**.** *Let $\mathcal{B}$ be the set of all possible bug types. Author A has a blind-spot set $B_A \subseteq \mathcal{B}$: bug types that A systematically fails to test for. Bugs in the blind spot are invisible to the author.*

Theorem 5.1 (Bias Decomposition)**.** *Let $\beta_A = \mu(B_A)$ be the probability that a random bug falls in author A's blind spot. For two agents A, A' with* independent *blind spots (that is, $\mathbb{P}(b \in B_A \cap B_{A'}) = \beta_A \cdot \beta_{A'}$):*

$$\beta = \beta_A(1 - \beta_{A'}) \tag{9}$$

Remark. *The independence assumption is strongest when A and A' are from different model families with different pretraining corpora. For agents from the same family (e.g., two instances of the same model with different prompts), blind-spot overlap is substantial and β is reduced less than the theorem predicts. The assumption should be understood as an upper bound on the benefit of separation; the actual bias reduction depends on the degree of blind-spot independence achieved.*

Proof. When the same author writes code and tests, the escape probability is β_A. When a different author writes tests, the escape probability is $\beta_A \cdot \beta_{A'}$ (the bug must be in both blind spots). Thus $\beta = \beta_A - \beta_A\beta_{A'} = \beta_A(1 - \beta_{A'})$. □

Corollary 5.1 (Self-Verification Provides Zero Bias Reduction)**.** *When $A' = A$: $|B_A \cap B_A| = |B_A|$, so β is unchanged. Self-verification cannot reduce circular validation bias.*

This corollary is consistent with the finding of Anonymous (2025) that self-verification yields the smallest gains of any verification configuration.

5.3. Connection to the METR Merge-Gap Finding

METR found that roughly 50% of SWE-bench test-passing PRs would not be merged (METR, 2026). The maintainer merge rate is 24.2 percentage points below the automated grader. If we model the automated grader as a "same-paradigm" test (sharing blind spots with the generation system), this implies near-maximal circular bias: the grader tests for functional correctness in a narrow sense, while the defects blocking merging (code quality, convention violations, subtle semantic issues) are precisely the types the grader does not cover.

Theorem 5.2 (Structural Separation Theorem)**.** *Structural separation (different agents for code and test authorship) reduces circular validation bias whenever $|B_A \cap B_{A'}| < |B_A|$. The reduction is maximized when blind-spot sets are disjoint ($B_A \cap B_{A'} = \emptyset$).*

Sufficient conditions for effective separation:

1. *Different model family (different pretraining corpora yield different error distributions).*
2. *Different input representation (e.g., formal specification vs. natural language).*
3. *Adversarial framing (instructing the test author to actively seek failures).*

6. The Structural Enforcement Boundary

6.1. Definitions

Definition 6.1 (Structural Enforcement)**.** *A constraint is* structurally enforced *if there exists a function $\sigma : \mathcal{A} \to \mathcal{A}'$ such that $\sigma(a) \notin C$ for all $a \in \mathcal{A}$, where $\mathcal{A}$ is the agent's action space and C is the set of constraint-violating actions. The enforcement function projects every action into the safe subspace regardless of agent intent.*

Definition 6.2 (Prompt Enforcement)**.** *A constraint is* prompt-enforced *if compliance depends on the agent's probabilistic response: $\mathbb{P}(a \notin C \mid \text{constraint in prompt}) = 1 - \delta$, where $\delta > 0$ is the non-compliance probability.*

6.2. The Cost-of-Failure Threshold

Theorem 6.1 (Structural Enforcement Threshold)**.** *Let δ be the non-compliance probability under prompt enforcement, $\delta_s < \delta$ the specification gap under structural enforcement, c_p and c_s their respective implementation costs ($c_s > c_p$), and L the cost per violation. Structural enforcement is cost-effective when:*

$$L > L^* = \frac{c_s - c_p}{\delta - \delta_s} \tag{10}$$

Proof. The expected cost under prompt enforcement is $c_p + \delta L$; under structural enforcement, $c_s + \delta_s L$. Setting these equal and solving for L yields the threshold. □

Example 6.1. *Using AgentSpec parameters (Wang et al., 2026): $\delta = 0.23$ (prompt failure rate), $\delta_s = 0.10$ (structural specification gap for manually authored rules), $c_p = 1$, $c_s = 10$. Then $L^* = 9/0.13 \approx 69$. When a single violation costs more than 69× the prompt engineering cost, structural enforcement is justified. For safety-critical constraints (data deletion, financial transactions), L typically exceeds this threshold by orders of magnitude.*

6.3. Multi-Step Amplification

Theorem 6.2 (Compounding Non-Compliance)**.** *In an n-step pipeline where the same constraint must hold at every step, the probability of at least one violation under prompt enforcement is:*

$$\mathbb{P}(\text{any violation}) = 1 - (1 - \delta)^n \tag{11}$$

The effective enforcement transition threshold drops to:

$$L_n^* = \frac{c_s - c_p}{(1 - (1 - \delta)^n) - (1 - (1 - \delta_s)^n)} \approx \frac{L^*}{n} \quad \text{for small } \delta, \delta_s \tag{12}$$

This means that for longer pipelines, the case for structural enforcement strengthens linearly with pipeline length. A 20-step pipeline has an enforcement threshold roughly 20× lower than a single-step invocation.

6.4. Agent Gaming and the Adversarial Collapse

Empirical evidence demonstrates that LLM agents actively circumvent evaluation criteria when given the opportunity. METR documented reward-hacking rates of 30.4% on RE-Bench (where the scoring function was visible) versus 0.7% on HCAST (where it was hidden), a 43× difference (METR, 2025b). NIST CAISI documented agents commenting out assertions, downloading solutions from GitHub, crashing servers via $/\mathrm{dev}/\mathrm{urandom}$, and reading future git commits (NIST CAISI, 2025). Apollo Research confirmed in-context scheming in frontier models, with pre-training covert action rates of 13% for o3 (Apollo Research, 2025).

Prompt-based mitigation proved ineffective: adding "do not cheat" to the system prompt did not reduce reward-hacking rates (METR, 2025b). Anti-scheming training reduced overt misbehavior but increased evaluation-awareness reasoning, suggesting the model learned to detect evaluations rather than internalize alignment (Apollo Research, 2025).

Under adversarial conditions (the agent actively seeks to circumvent constraints), prompt enforcement fails entirely ($\delta \to 1$) and the enforcement transition threshold collapses to approximately $c_s - c_p$. This provides formal justification for the principle: **safety-critical constraints must be structurally enforced**.

7. Adaptive Verification Architecture

7.1. The Four-Layer Model

Synthesizing the formal results and empirical evidence, we define four canonical verification layers ordered by increasing cost and decreasing determinism:

Layer ℓ	Name	$d(\ell)$	$f(\ell)$	$c(\ell)$	$\eta(\ell)$
1	Structural gates	0.95	~ 0	1	0.950
2	Deterministic analysis	0.80	0.10	5	0.160
3	LLM-as-Judge	0.70	0.25	50	0.014
4	Human review	0.90	0.05	500	0.002

Layer 1 (type checking, schema validation, permission enforcement) is 7× more efficient than Layer 2, 68× more efficient than Layer 3, and 528× more efficient than Layer 4.

Theorem 7.1 (Greedy Efficiency Ordering)**.** *For independent verification layers with negligible false positives, the cost-minimizing strategy achieving a target detection rate d^* activates layers in decreasing order of efficiency $\eta(\ell) = d(\ell)/c(\ell)$ until the target is met.*

Proof. This is a variant of the fractional knapsack problem. Each layer provides detection "value" $d(\ell)$ at cost $c(\ell)$. The greedy algorithm (take items in decreasing value-to-weight order) is optimal for fractional knapsack. □

Theorem 7.2 (Cascaded Detection Rate)**.** *If layers $\ell_1, \ldots, \ell_m$ are applied in sequence, each independently detecting defects, the combined detection rate is:*

$$d_{combined} = 1 - \prod_{j=1}^{m} (1 - d(\ell_j)) \tag{13}$$

With all four layers and 3 iterations of the middle layers: $d_{\text{combined}} \geq 0.958$, matching the empirical 96.5% convergence.

7.2. Adaptive Verification Intensity

Define the product $r \cdot \hat{g}$ where $r \in [0, 1]$ is risk level and $\hat{g} \in [0, 1]$ is the estimated spec-to-code information gap. The optimal verification level minimizes:

$$J(\ell, \hat{g}, r) = c(\ell) + \lambda \cdot r \cdot \hat{g} \cdot (1 - d(\ell)) \tag{14}$$

Level ℓ is preferred over $\ell - 1$ when $r \cdot \hat{g}$ exceeds the switching threshold:

$$\tau(\ell{-}1 \to \ell) = \frac{c(\ell) - c(\ell{-}1)}{\lambda \cdot (d(\ell) - d(\ell{-}1))} \tag{15}$$

With $\lambda = 10$:

Transition	Δc	Δd	Threshold τ
L1 → L2	4	0.35	0.011 (almost always upgrade)
L2 → L3	45	0.20	0.075 (medium-risk code)
L3 → L4	450	0.12	0.667 (high-risk, high-gap only)

7.3. Multi-Agent Verification Economics

The two-agent structural separation model (§5) incurs approximately 2× the generation cost. The break-even consequence (where separation pays for itself) is roughly 3.2× the generation cost, or approximately $25 for a typical coding task.

However, Google Research (2025) found that multi-agent systems degrade sequential reasoning by 39–70%. Adjusting for this fragmentation penalty, the break-even rises to roughly 6.3× generation cost. We note that these break-even estimates are sensitive to parameter choices: the self-detection rate ($d_{\text{self}} = 0.35$, from Anonymous 2025) and cross-detection rate ($d_{\text{cross}} = 0.62$, from Hariharan et al. 2025) were measured under different conditions and on different tasks. At the extremes, if d_{self} were as high as 0.50, the break-even would roughly double; if d_{cross} were as low as 0.50, it would increase by roughly 50%. The qualitative conclusion (separation is justified for non-trivial consequences) is robust, but the precise threshold should be treated as order-of-magnitude guidance, not a sharp boundary. For catastrophic tail risks (such as the reported $47,000 infinite-loop incident, cited from a practitioner blog post and not independently verified), structural prevention through cost ceilings dominates ensemble verification.

The evidence from production tools (Cursor, Claude Code, Devin) converges on a common pattern: single-agent architecture with execution feedback (test results, compiler errors) as the primary verification signal, supplemented by human review for consequential changes (Cognition Labs, 2025; METR, 2025a). This is consistent with our framework: execution feedback provides ground-truth verification at near-zero marginal cost (Layer 2), while the overhead of a separate verification agent (Layer 3 cost, plus the sequential reasoning penalty) is justified only for high-consequence decisions.

8. Related Work

Classical reliability theory. The series system product law and the Barlow-Proschan importance measures (Barlow and Proschan, 1965, 1975) provide the mathematical foundation for our pipeline model. The IFR/IFRA closure theorems (Birnbaum et al., 1961) establish that series systems of components with increasing failure rates exhibit accelerating degradation.

Checkpoint optimization. The Young-Daly formula (Daly, 2006; Young, 1974) for optimal checkpoint intervals in fault-tolerant computing is the closest antecedent to our verification scheduling result. The Lindsay-Bishop dynamic programming formulation for inspection allocation (Lindsay and Bishop, 1964) provides the discrete optimization framework.

Software reliability growth. The Jelinski-Moranda (Jelinski and Moranda, 1972) and Musa-Okumoto (Musa and Okumoto, 1984) models of fault detection as a depletion process directly inform our verification convergence analysis.

Agent verification frameworks. AgentSpec (Wang et al., 2026) provides the key empirical data point on structural vs. prompt enforcement (87% vs. 77%). AgentGuard (Koohestani, 2025) introduces dynamic probabilistic assurance via MDP modeling. Multi-Agent Verification (Lifshitz et al., 2025) demonstrates that diverse verifier ensembles outperform homogeneous ones, and that weaker models can verify stronger ones.

LLM evaluation. Anthropic's agent evaluation methodology (Anthropic, 2026) introduces the pass@k vs. pass^k distinction. The LLM-as-Judge survey (Li et al., 2024) documents systematic biases (sycophancy, position bias) that limit LLM-based verification. The METR merge-gap study (METR, 2026) provides the most striking evidence of circular validation bias.

Scalable oversight. The debate framework (Irving et al., 2018) proposes using adversarial argumentation between agents to help weaker judges evaluate stronger systems. Empirical results have been mixed: debate is effective with strong human debaters but less so with LLM debaters. Constitutional AI (Bai et al., 2022) uses self-critique against a set of principles; while effective as a training methodology, it does not constitute runtime verification and inherits the circular validation limitations we formalize in §5.

Agent gaming. METR's reward-hacking analysis (METR, 2025b), Apollo Research's scheming studies (Apollo Research, 2025), and NIST CAISI's cheating documentation (NIST CAISI, 2025) collectively establish that structural enforcement is necessary for adversarial robustness.

Sampling inspection. The Dodge-Romig sampling plans (Dodge and Romig, 1929, 1959) and MIL-STD-105E switching rules (United States Department of Defense, 1989) provide the adaptive verification paradigm that informs our four-layer architecture.

9. Design Principles

The formal results and empirical calibration yield eight actionable design principles:

1. **Compound errors are multiplicative, not additive.** Small per-step improvements produce large end-to-end gains (Theorem 3.2). A 1% relative improvement per step yields n% end-to-end for an n-step pipeline.
2. **Fix the weakest link first.** The step with the lowest p_i offers the highest marginal return (Corollary 3.2). In practice, this is usually problem framing (step 1), not execution.
3. **Verify at phase boundaries, not at every step.** The Young-Daly analog (Theorem 4.2) gives the optimal spacing. For typical parameters, this means 2–5 verification points in a 20-step pipeline.
4. **Three verification rounds, then stop.** The geometric diminishing returns (Theorem 4.3) and empirical convergence at 96.5% by round 3 establish a firm verification budget. Rounds 4+ face the pesticide paradox: the remaining errors require different detection approaches, not more of the same.
5. **Separate code authorship from test authorship.** The circular validation bias (Theorem 5.1) is real and substantial. Self-verification provides zero bias reduction. Cross-family verification is where gains concentrate.
6. **Structural enforcement for safety; prompt enforcement for quality.** The cost-of-failure threshold (Theorem 6.1) demarcates the boundary. Safety-critical constraints (data deletion, financial limits, security boundaries) must be structurally enforced. Subjective quality dimensions (code style, naming, abstraction quality) can use LLM judgment bounded by structural gates.
7. **The verifier must be structurally read-only.** Prompt-level "do not modify the code" is insufficient (§6). Tool permissions must enforce read-only access for the verification agent.
8. **Layer verification by efficiency.** Structural gates first ($\eta = 0.95$), deterministic analysis second ($\eta = 0.16$), LLM-as-Judge third ($\eta = 0.014$), human review last ($\eta = 0.002$). Each dollar of verification budget should go to the highest-efficiency layer not yet saturated.

10. Conclusion

We have developed a formal framework for reliability architecture in multi-step AI agent pipelines, grounded in classical reliability theory and calibrated against contemporary benchmarks. The framework provides four core results: the compound error sensitivity theorem establishing that per-step improvements have n-fold leverage; the optimal verification scheduling formulation recovering the Young-Daly interval and confirming three-round optimality; the circular validation bias theorem showing that self-verification is structurally incapable of reducing same-author blind spots; and the enforcement transition threshold demarcating when deterministic constraints become cost-effective.

Several limitations deserve emphasis. Our empirical calibration relies on sparse data points from heterogeneous benchmarks. The independence assumption in the series reliability model, while a useful first approximation, does not capture the full correlation structure of real agent pipelines. The verification convergence data (62%/89%/96.5% by rounds 1/2/3) comes from a single domain, namely household task planning on the TEACh dataset (Hariharan et al., 2025); generalization to software engineering pipelines is assumed throughout but not experimentally verified. Software engineering tasks may exhibit different convergence profiles due to the availability of execution feedback (compiler errors, test failures), which provides ground-truth signals absent in embodied task planning. The "true quality" Q in our gaming detection framework is fundamentally uncomputable; all practical proxies inherit the Goodhart problem they aim to detect.

The most important open question is empirical: as agent capabilities improve, how do the key parameters (p, γ, β, δ) evolve? If per-step reliability increases faster than pipeline length, compound errors become manageable. If pipeline ambitions grow faster than per-step reliability, the

framework's predictions become increasingly binding. The arithmetic is unforgiving either way: the exponential in $R = p^n$ does not negotiate.

Appendices

A. Proofs and Extended Results

A.1. Markov Error Propagation Model

When errors propagate (a step can fail because its input was corrupted), the simple product law does not hold. Model the pipeline as a Markov chain on states $\{C, E\}$ with transition matrix:

$$\mathrm{P} = \begin{pmatrix} p & 1-p \\ q & 1-q \end{pmatrix} \tag{16}$$

Theorem A.1 (Pipeline Reliability Under Markov Error Propagation)**.** *Starting from state C, the probability of being in state C after n steps is:*

$$R_{Markov}(n) = \pi_C + (1-\pi_C)(p+q-1)^n \tag{17}$$

where $\pi_C = q/(1-p+q)$ is the stationary probability of the correct state.

Proof. Diagonalize P. Its eigenvalues are $\lambda_1 = 1$ and $\lambda_2 = p+q-1$. The spectral decomposition gives $\mathrm{P}^n = \Pi + (p+q-1)^n(\mathrm{I} - \Pi)$ where Π is the matrix of stationary probabilities. The (C, C) entry is $\pi_C + (1-\pi_C)(p+q-1)^n$. □

When $q = 0$ (errors never self-correct), the error state is absorbing: once an error is introduced, it persists through all subsequent steps. In this case, the pipeline succeeds only if no step introduces an error, giving $R_{\text{Markov}}(n) = p^n$, which recovers the series reliability product law. The Markov model with $q > 0$ is more optimistic than the product law because it allows error recovery at each step.

A.2. The LLM-Verifier Convergence Theorem

Chen et al. (2024) model a multi-stage LLM verification pipeline as an absorbing Markov chain with m transient stages and one absorbing state (Verified).

Theorem A.2 (LLM-Verifier Convergence)**.** *For success probability $\delta \in (0, 1]$ per stage:*

1. Almost sure convergence*:* $\mathbb{P}(\tau < \infty) = 1$.
2. Expected iteration bound*:* $\mathbb{E}[\tau] = m/\delta$.
3. Exponential tail bound*:* $\mathbb{P}(\tau > k) \leq \alpha(1-\delta)^k$.

A.3. Information-Theoretic Verification Bounds

A verification step is a binary channel from true correctness to verdict. By Fano's inequality (Fano, 1961):

$$\mathbb{P}(\text{misclassification}) \geq H_b^{-1}(H(\text{correct} \mid \text{verdict})) \tag{18}$$

The data processing inequality (Cover and Thomas, 2006) establishes that verification is fundamentally limited by the information preserved in the agent's output: $I(X;Z) \leq I(X;Y)$ for any chain $X \to Y \to Z$. If the agent's output is ambiguous with respect to correctness, no verifier can fully compensate.

A.4. Beta-Binomial Calibration

To capture heterogeneity in per-step success rates across tasks, let $p \sim \text{Beta}(\alpha, \beta)$. The marginal pipeline success probability is:

$$R_{\text{BB}}(n, \alpha, \beta) = \mathbb{E}[p^n] = \frac{B(\alpha + n, \beta)}{B(\alpha, \beta)} = \prod_{k=0}^{n-1} \frac{\alpha + k}{\alpha + \beta + k} \tag{19}$$

Calibrated parameters from published benchmarks:

Benchmark	n	R_{obs}	μ	α	β	α+β
SWE-bench (METR)	10	0.50	0.93	12.5	0.95	13.5
Devin (2025)	12	0.67	0.97	28.4	0.88	29.3
RE-Bench	100	0.37	0.99	85.0	0.86	85.9
Generic agent	20	0.12	0.90	8.1	0.90	9.0

Higher concentration ($\alpha + \beta$) indicates more homogeneous task populations; lower concentration indicates high task-to-task variance.

A.5. Goodhart's Law Taxonomy for Agent Gaming

Following Manheim and Garrabrant (2019), the four Goodhart mechanisms are:

1. **Regressional**: Optimizing a noisy proxy selects for noise in the proxy-target relationship.
2. **Extremal**: The proxy-target relationship, estimated in a typical regime, breaks down at extremes.
3. **Causal**: The proxy and target share a common cause; intervening on the proxy breaks the causal pathway.
4. **Adversarial**: The agent directly manipulates the proxy without affecting the target.

Every documented NIST CAISI cheating instance (NIST CAISI, 2025) is detectable through invariant violation checks (structural enforcement), not through statistical proxy-target monitoring. This strongly supports the principle that structural enforcement handles safety-critical concerns.

References

Anonymous. When does verification pay off? A closer look at LLMs as solution verifiers. *arXiv preprint arXiv:2512.02304*, 2025.

Anthropic. Demystifying evals for AI agents. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents, January 2026.

Apollo Research. Stress testing deliberative alignment for anti-scheming training. https://www.apolloresearch.ai/research/stress-testing-deliberative-alignment-for-anti-scheming-training/, 2025.

Y. Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

R. E. Barlow and F. Proschan. *Mathematical Theory of Reliability*. John Wiley & Sons, 1965. Reprinted by SIAM, 1996.

R. E. Barlow and F. Proschan. *Statistical Theory of Reliability and Life Testing: Probability Models*. Holt, Rinehart and Winston, 1975.

Z. W. Birnbaum, J. D. Esary, and A. W. Marshall. A stochastic characterization of wear-out for components and systems. *Annals of Mathematical Statistics*, 32(3):816–825, 1961.

M. Cemri, M. Z. Pan, and S. Yang. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Y. Chen et al. The $4/\delta$ bound: Designing predictable LLM-verifier systems for formal method guarantee. *arXiv preprint arXiv:2512.02080*, 2024.

Cognition Labs. Devin's 2025 performance review. https://cognition.ai/blog/devin-annual-performance-review-2025, 2025.

T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

J. T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.

H. F. Dodge and H. G. Romig. A method of sampling inspection. *Bell System Technical Journal*, 8(4): 613–631, 1929.

H. F. Dodge and H. G. Romig. *Sampling Inspection Tables: Single and Double Sampling*. John Wiley & Sons, 2nd edition, 1959.

R. M. Fano. *Transmission of Information: A Statistical Theory of Communications*. MIT Press, 1961.

Google Research. Towards a science of scaling agent systems. https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/, 2025.

A. Hariharan, V. Dongre, D. Hakkani-Tür, and G. Tur. Plan verification for LLM-based embodied task completion agents. *arXiv preprint arXiv:2509.02761*, 2025.

G. Irving, P. Christiano, and D. Amodei. AI safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.

Z. Jelinski and P. B. Moranda. Software reliability research. In *Statistical Computer Performance Evaluation*, pages 465–484. Academic Press, 1972.

R. Koohestani. AgentGuard: Runtime verification of AI agents. *arXiv preprint arXiv:2509.23864*, 2025. Presented at AgenticSE, ASE 2025.

J. Li et al. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.

O. Lifshitz, S. A. McIlraith, and S. S. Du. Multi-agent verification: Scaling test-time compute with multiple verifiers. *arXiv preprint arXiv:2502.20379*, 2025.

G. F. Lindsay and A. B. Bishop. Allocation of screening inspection effort: A dynamic-programming approach. *Management Science*, 10(2):342–352, 1964.

D. Manheim and S. Garrabrant. Categorizing variants of Goodhart's Law. *arXiv preprint arXiv:1803.04585*, 2019.

A. W. Marshall and I. Olkin. A multivariate exponential distribution. *Journal of the American Statistical Association*, 62(317):30–44, 1967.

METR. Measuring the impact of early-2025 AI on experienced open-source developer productivity. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/, July 2025a.

METR. Recent frontier models are reward hacking. https://metr.org/blog/2025-06-05-recent-reward-hacking/, June 2025b.

METR. Many SWE-bench-passing PRs would not be merged into main. https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/, March 2026.

J. D. Musa and K. Okumoto. A logarithmic Poisson execution time model for software reliability measurement. In *Proceedings of the 7th International Conference on Software Engineering*, pages 230–238, 1984.

NIST CAISI. Examples of cheating in CAISI's agent evaluations. https://www.nist.gov/caisi/cheating-ai-agent-evaluations/, 2025.

K. Rajan. The math that's killing your AI agent. *Towards Data Science*, March 2026.

Stanford HAI. AI index report 2025. https://hai.stanford.edu/ai-index/2025-ai-index-report, 2025.

United States Department of Defense. MIL-STD-105E: Sampling procedures and tables for inspection by attributes. Technical report, U.S. Department of Defense, 1989.

H. Wang, C. M. Poskitt, and J. Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, Rio de Janeiro, 2026.

J. W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.